Blood Transfusion

Made by:

- Irvan Ilahibaks - 8081847
- Timon Baris - 2173182
- Waling van Dongen – 1176226

TA: Sita Newer

Vak: Data Analytics

13-10-2024

Task 1.1: Exploring dataset

```
In [4]: print(btData.to_string())
```

	months_since_last_donation	total_number_of_donations	total_blood_donated	months_since_first_donation	class
0	2.0	50.0	12500.0	98.0	1
1	0.0	13.0	3250.0	28.0	1
2	1.0	16.0	4000.0	35.0	1
3	2.0	20.0	5000.0	45.0	1
4	1.0	24.0	6000.0	77.0	0
5	4.0	4.0	1000.0	4.0	0
6	2.0	7.0	1750.0	14.0	1
7	1.0	12.0	3000.0	35.0	0
8	2.0	9.0	2250.0	22.0	1
9	5.0	46.0	11500.0	98.0	1
10	4.0	23.0	5750.0	58.0	0
11	0.0	3.0	750.0	4.0	0
12	2.0	10.0	2500.0	28.0	1
13	1.0	13.0	3250.0	47.0	0
14	2.0	6.0	1500.0	15.0	1
15	2.0	5.0	1250.0	11.0	1
16	2.0	14.0	3500.0	48.0	1
17	2.0	15.0	3750.0	49.0	1

```
#1.1 Get data on the screen
import pandas as pd

btData =  pd.read_csv('blood_transfusion.csv') #Initialize data
```

```
btData.info() #vieuwing Data Types
```

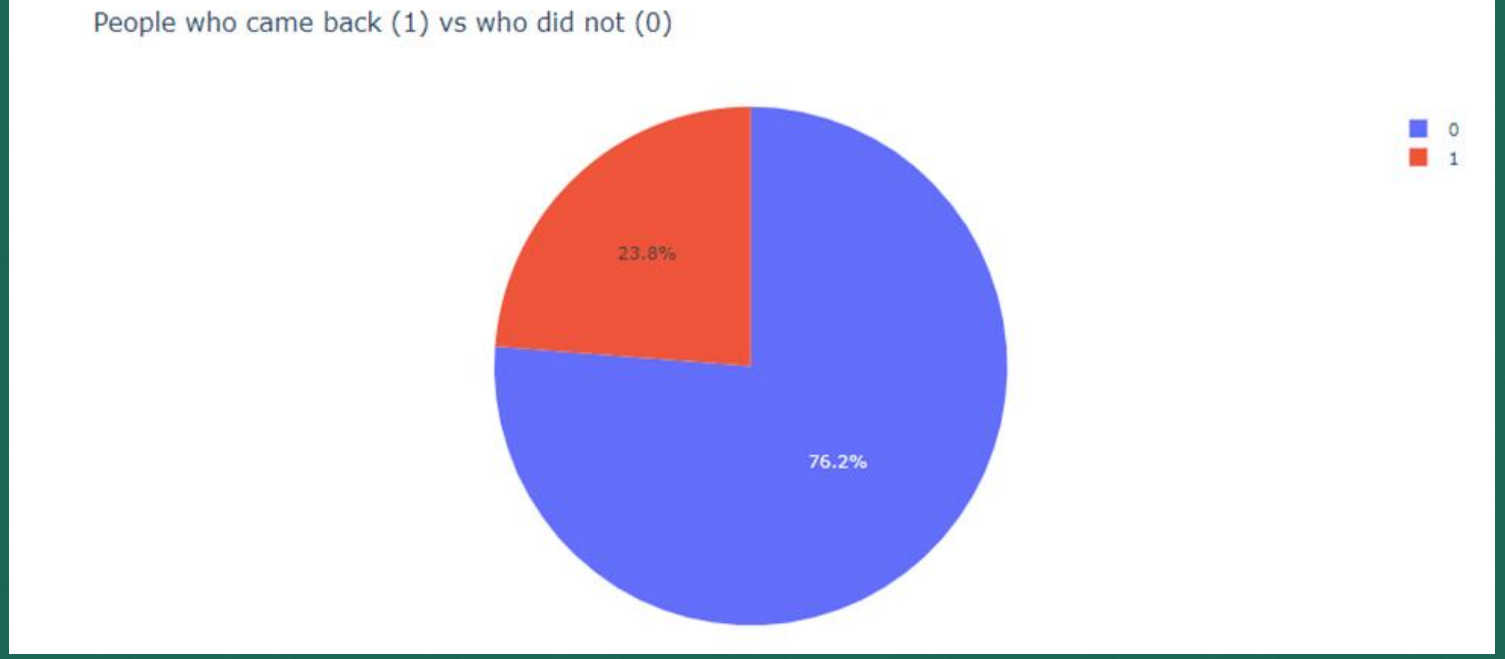
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 748 entries, 0 to 747
Data columns (total 5 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   months_since_last_donation  748 non-null    float64
 1   total_number_of_donations   748 non-null    float64
 2   total_blood_donated         748 non-null    float64
 3   months_since_first_donation  748 non-null    float64
 4   class                       748 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 29.3 KB
```

```
btData.describe() #summary statistics
```

	months_since_last_donation	total_number_of_donations	total_blood_donated	months_since_first_donation	class
count	748.000000	748.000000	748.000000	748.000000	748.000000
mean	9.506684	5.514706	1378.676471	34.282086	0.237968
std	8.095396	5.839307	1459.826781	24.376714	0.426124
min	0.000000	1.000000	250.000000	2.000000	0.000000
25%	2.750000	2.000000	500.000000	16.000000	0.000000
50%	7.000000	4.000000	1000.000000	28.000000	0.000000
75%	14.000000	7.000000	1750.000000	50.000000	0.000000
max	74.000000	50.000000	12500.000000	98.000000	1.000000

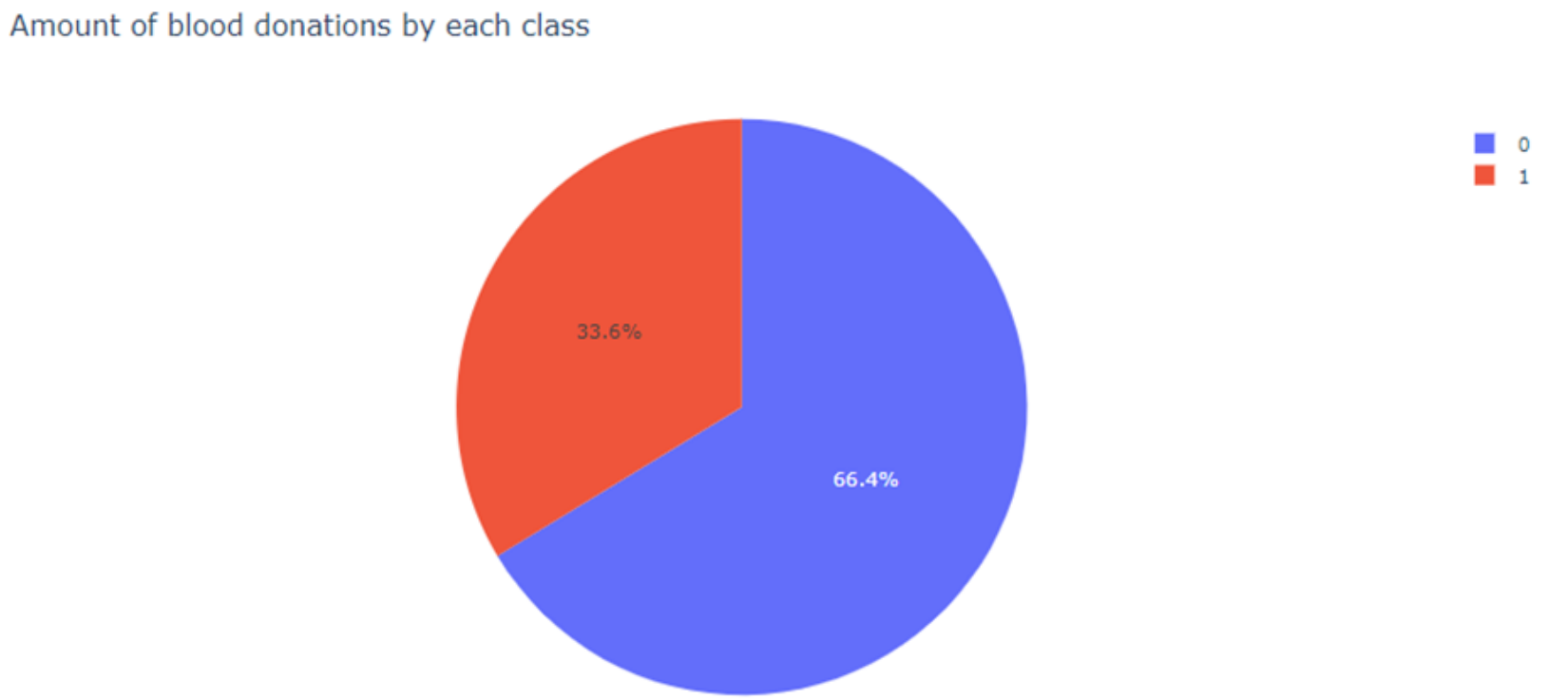
Task 1.1: Data visualisation

```python
import plotly.express as px

#counting how many people there are of each class
peopleCount = btData['class'].value_counts().reset_index()

#naming the columns
peopleCount.columns = ['class', 'count']

# Create the pie chart using the counts
figpie = px.pie(peopleCount, values='count', names='class', title='People who came back (1) vs who did not (0)')
figpie.show()

```



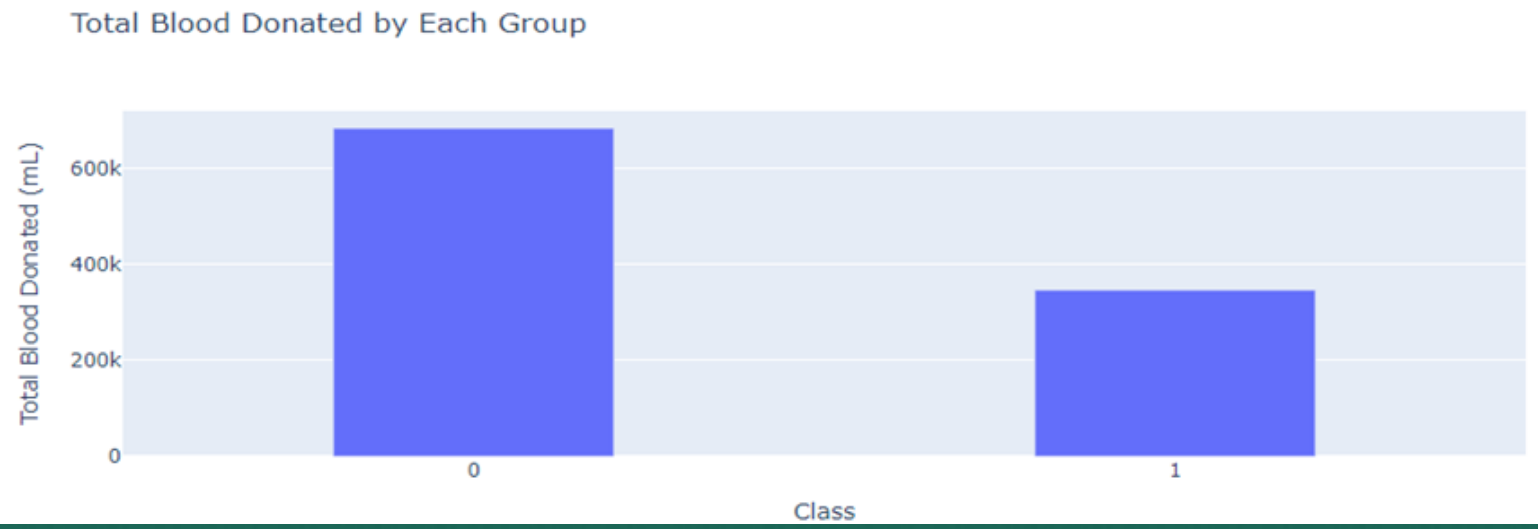
Task 1.1: Data visualisation

```
# Grouping the data by class and summing the total blood donations
totalBloodDon = btData.groupby('class')['total_number_of_donations'].sum().reset_index()
totalBloodDon.columns = ['class', 'count']

figpie = px.pie(totalBloodDon, values='count', names='class', title='Amount of blood donations by each class')
figpie.show()

```



Task 1.1: Data visualisation

```python
# Grouping the data by class and summing the total blood donated
totalBlood = btData.groupby('class')['total_blood_donated'].sum().reset_index()

# Create the bar chart
fig = px.bar(totalBlood, x='class',
             y='total_blood_donated',
             title="Total Blood Donated by Each Group",
             labels={'class': 'Class', 'total_blood_donated': 'Total Blood Donated (mL)'},
            )

fig.update_layout(
    xaxis=dict(tickvals=[0, 1], range=[-0.5, 1.5]),
    height=400,
    bargap=0.6)

fig.show()

```



Task 1.1: Data visualisation

```
averageBlood = btData.groupby('class')['total_blood_donated'].mean().reset_index()

fig = px.bar(averageBlood, x='class', y='total_blood_donated', title="Average Blood Donated by Each Class",
             labels={'class': 'Class', 'total_blood_donated': 'Average Blood Donated (ml)'})

fig.update_layout(
    xaxis=dict(tickvals=[0, 1], range=[-0.5, 1.5]),
    bargap=0.6
)
```

Task 2: Preprocessing

```
#Task 2: Preprocessing

btData.isnull().sum() #The data below shows that every cell of data has data
```

```
months_since_last_donation     0
total_number_of_donations      0
total_blood_donated            0
months_since_first_donation    0
class                          0
dtype: int64
```

```
from sklearn.preprocessing import StandardScaler

features = btData.drop('class', axis=1)

standardSC = StandardScaler()

standardFeatures = standardSC.fit_transform(features)

btSData = pd.DataFrame(standardFeatures, columns=features.columns)

print("StandarScaler Features:")
print(btSData.head())
```

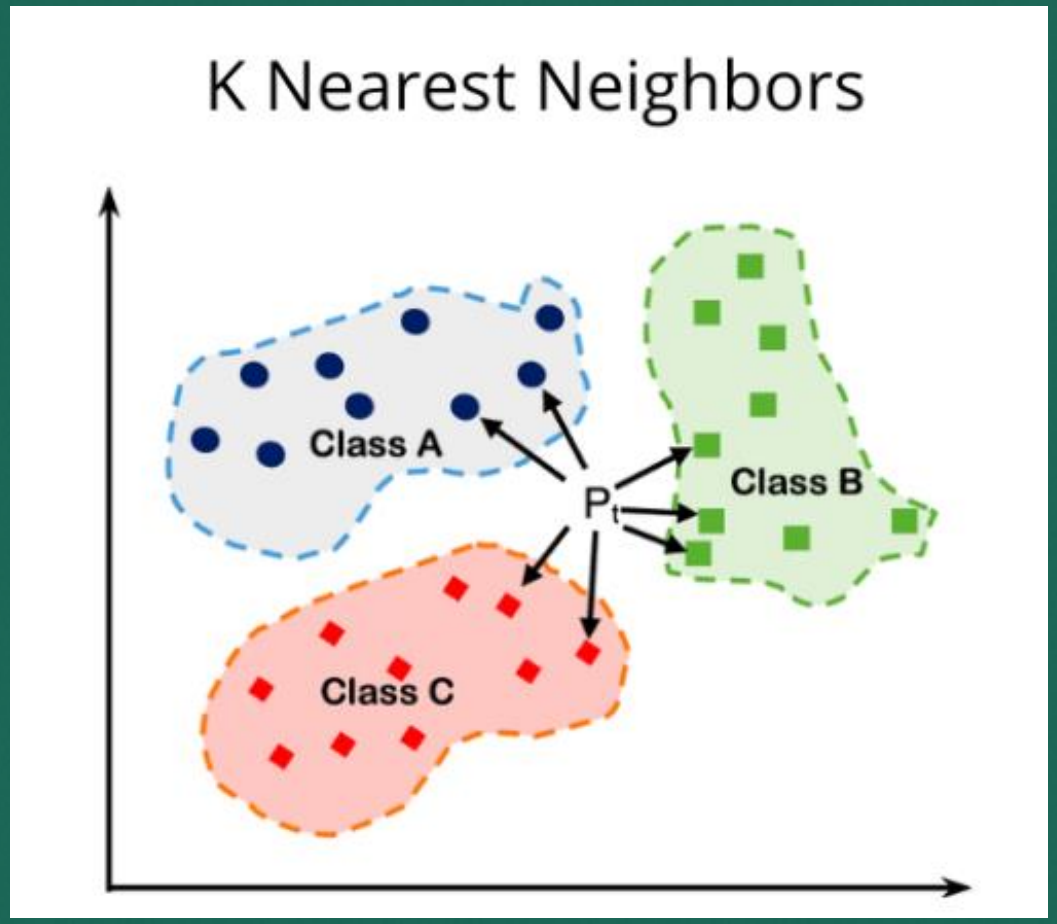
```
StandarScaler Features:
   months_since_last_donation  total_number_of_donations  total_blood_donated  \
0                   -0.927899                   7.623346             7.623346
1                   -1.175118                   1.282738             1.282738
2                   -1.051508                   1.796842             1.796842
3                   -0.927899                   2.482313             2.482313
4                   -1.051508                   3.167784             3.167784

   months_since_first_donation
0                     2.615633
1                    -0.257881
2                     0.029471
3                     0.439973
4                     1.753579
```

Task 3: Creating a train and test set

```python
#3.1: Creating a train and test set

from sklearn.model_selection import train_test_split

# Separate features and the target class
X = btSData
y = btData['class']  # Target

# Create 80-20 train-test split
X_train_80, X_test_80, y_train_80, y_test_80 = train_test_split(X, y, test_size=0.2, random_state=

# Create 70-30 train-test split
X_train_70, X_test_70, y_train_70, y_test_70 = train_test_split(X, y, test_size=0.3, random_state=

# Compare the results of the trained set to the test set and round the result to 3 decimales to ma
print(f"80/20 train test split\nTrain set {y_train_80.value_counts(normalize=True).round(3).to_dic
print(f"70/30 train test split\nTrain set {y_train_70.value_counts(normalize=True).round(3).to_dic

```

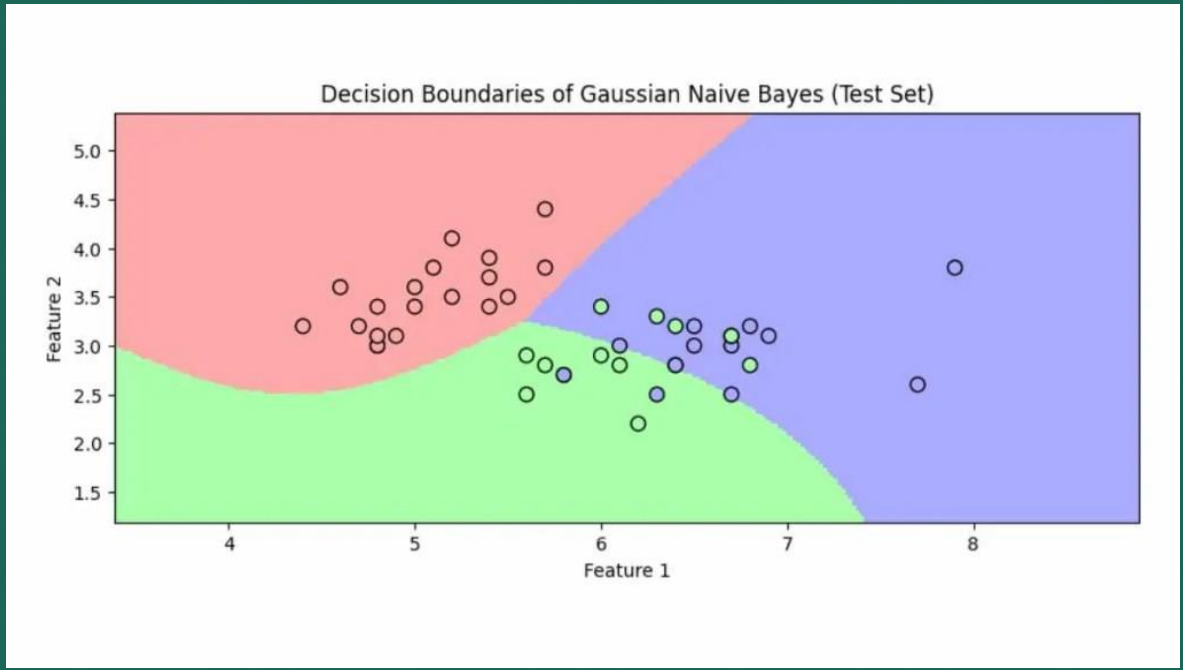
```
80/20 train test split
Train set {0: 0.764, 1: 0.236}
Test set {0: 0.753, 1: 0.247}

70/30 train test split
Train set {0: 0.774, 1: 0.226}
Test set {0: 0.733, 1: 0.267}
```

Task 4.1: Classification algorithms – KNN



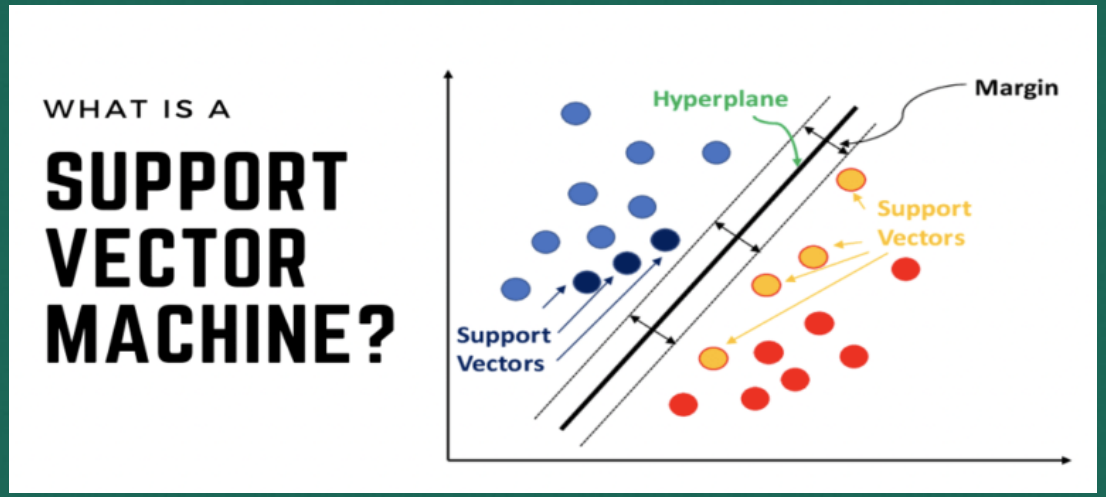

```
KNN Classifier 80/20:
Test sample 1: Predicted = 0, Actual = 0
Test sample 2: Predicted = 0, Actual = 0
Test sample 3: Predicted = 0, Actual = 0
Test sample 4: Predicted = 1, Actual = 0
Test sample 5: Predicted = 0, Actual = 0
Test sample 6: Predicted = 1, Actual = 0
Test sample 7: Predicted = 0, Actual = 0
Test sample 8: Predicted = 0, Actual = 0
Test sample 9: Predicted = 0, Actual = 0
Test sample 10: Predicted = 0, Actual = 0
Test sample 11: Predicted = 1, Actual = 0
Test sample 12: Predicted = 1, Actual = 0
Test sample 13: Predicted = 0, Actual = 0
Test sample 14: Predicted = 0, Actual = 1
Test sample 15: Predicted = 0, Actual = 0
Test sample 16: Predicted = 0, Actual = 0
Test sample 17: Predicted = 1, Actual = 0
Test sample 18: Predicted = 0, Actual = 0
Test sample 19: Predicted = 0, Actual = 0
Test sample 20: Predicted = 0, Actual = 1
Test sample 21: Predicted = 1, Actual = 1
Test sample 22: Predicted = 1, Actual = 1
Test sample 23: Predicted = 0, Actual = 0
Test sample 24: Predicted = 0, Actual = 0
Test sample 25: Predicted = 0, Actual = 0
Test sample 26: Predicted = 0, Actual = 0
Test sample 27: Predicted = 0, Actual = 0
Test sample 28: Predicted = 0, Actual = 1
Test sample 29: Predicted = 1, Actual = 0
Test sample 30: Predicted = 1, Actual = 0
Test sample 31: Predicted = 1, Actual = 1
Test sample 32: Predicted = 0, Actual = 0
Test sample 33: Predicted = 0, Actual = 0
Test sample 34: Predicted = 0, Actual = 1
Test sample 35: Predicted = 1, Actual = 0
Test sample 36: Predicted = 1, Actual = 0
Test sample 37: Predicted = 1, Actual = 0
Test sample 38: Predicted = 0, Actual = 0
Test sample 39: Predicted = 0, Actual = 0
Test sample 40: Predicted = 0, Actual = 0
Test sample 41: Predicted = 0, Actual = 1
Test sample 42: Predicted = 0, Actual = 0
Test sample 43: Predicted = 0, Actual = 0
Test sample 44: Predicted = 0, Actual = 0
Test sample 45: Predicted = 1, Actual = 0
Test sample 46: Predicted = 0, Actual = 0
Test sample 47: Predicted = 0, Actual = 0
Test sample 48: Predicted = 0, Actual = 0
Test sample 49: Predicted = 0, Actual = 1
Test sample 50: Predicted = 0, Actual = 0
```

Task 4.2: Classification algorithms – Naive Bayes



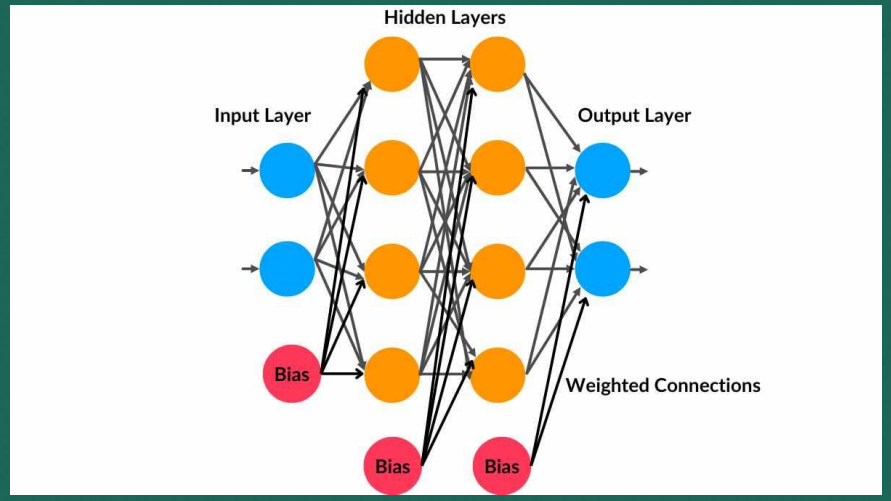

```python
#Task 4.2: Naive Bayes Classifier - 80/20 set & 70/30 set

from sklearn.naive_bayes import GaussianNB

# Initialize the Naive Bayes classification and train it on the 80/20 set
gnbClass80 = GaussianNB()
gnbClass80.fit(X_train_80, y_train_80)

# Initialize the Naive Bayes classification and train it on the 70/30 set
gnbClass70 = GaussianNB()
gnbClass70.fit(X_train_70, y_train_70)

# Select a few cases from the test set randomly
randomSamplesNBC = np.random.choice(X_test_80.index, 50, replace=False)
X_test_samplesNBC = X_test_80.loc[randomSamplesNBC]
y_test_samplesNBC = y_test_80.loc[randomSamplesNBC]

randomSamplesNBC2 = np.random.choice(X_test_70.index, 50, replace=False)
X_test_samplesNBC2 = X_test_70.loc[randomSamplesNBC2]
y_test_samplesNBC2 = y_test_70.loc[randomSamplesNBC2]


# Predict on the selected test samples
gnbPredictions = gnbClass80.predict(X_test_samplesNBC)
gnbPredictions2 = gnbClass70.predict(X_test_samplesNBC2)


# Compare predictions with actual labels
print("Naive Bayes Classifier Predictions 80/20:")
for i in range(50):
    print(f"Test sample {i+1}: Predicted = {gnbPredictions[i]}, Actual = {y_test_samplesNBC.iloc[i]}")

# Compare predictions with actual labels
print("\nNaive Bayes Classifier Predictions 70/30:")
for i in range(50):
    print(f"Test sample {i+1}: Predicted = {gnbPredictions2[i]}, Actual = {y_test_samplesNBC2.iloc[i]}")

```

Task 4.3: Classification algorithms – Support Vector




```python
#Task 4.3: Support Vector Classifier - 80/20 set & 70/30 set

from sklearn.svm import SVC
import numpy as np

# Initialize the Support Vector Classifier with class weights to handle imbalanced classes
svcData = SVC(class_weight='balanced')
svcData2 = SVC(class_weight='balanced')

# Train the model on the original 80-20 train set (no scaling)
svcData.fit(X_train_80, y_train_80)

# Train the model on the original 70-30 train set (no scaling)
svcData2.fit(X_train_70, y_train_70)

# Randomly select 5 samples from the test set
randomSamplesSVC = np.random.choice(X_test_80.index, 50, replace=False)
randomSamplesSVC2 = np.random.choice(X_test_70.index, 50, replace=False)

# Select the corresponding samples from the original test set
X_test_samplesSVC = X_test_80.loc[randomSamplesSVC]
y_test_samplesSVC = y_test_80.loc[randomSamplesSVC]

X_test_samplesSVC2 = X_test_70.loc[randomSamplesSVC2]
y_test_samplesSVC2 = y_test_70.loc[randomSamplesSVC2]

# Make predictions on the selected test samples
svcPredictions = svcData.predict(X_test_samplesSVC)
svcPredictions2 = svcData2.predict(X_test_samplesSVC2)

# Compare predictions with actual labels
print("SVC Classifier Predictions:")
for i in range(len(randomSamplesSVC)):
    print(f"Test sample {i+1}: Predicted = {svcPredictions[i]}, Actual = {y_test_samplesSVC.iloc[i]}")

print("\nSVC Classifier Predictions:")
for i in range(len(randomSamplesSVC2)):
    print(f"Test sample {i+1}: Predicted = {svcPredictions2[i]}, Actual = {y_test_samplesSVC2.iloc[i]}")

```

Task 4.4: Classification algorithms – Multilayer Perceptron




```python
#Task 4.4: Multilayer Perceptron (Neural Network) Classifier - 80/20 & 70/30 set

from sklearn.neural_network import MLPClassifier
import numpy as np

# Initialize the Multilayer Perceptron model
mlpData = MLPClassifier(max_iter=1000, random_state=42)
mlpData2 = MLPClassifier(max_iter=1000, random_state=42)

# Train the model on the original (non-scaled) 80-20 train set
mlpData.fit(X_train_80, y_train_80)

# Train the model on the original (non-scaled) 80-20 train set
mlpData2.fit(X_train_70, y_train_70)

# Randomly select 5 indices from the test set
randomSamplesMLP = np.random.choice(X_test_80.index, 50, replace=False)
randomSamplesMLP2 = np.random.choice(X_test_70.index, 50, replace=False)

# Select the corresponding non-scaled samples from X_test_20
X_test_samplesMLP = X_test_80.loc[randomSamplesMLP]
y_test_samplesMLP = y_test_80.loc[randomSamplesMLP]

# Select the corresponding non-scaled samples from X_test_30
X_test_samplesMLP2 = X_test_70.loc[randomSamplesMLP2]
y_test_samplesMLP2 = y_test_70.loc[randomSamplesMLP2]

# Make predictions on the selected test samples
mlpPredictions = mlpData.predict(X_test_samplesMLP)
mlpPredictions2 = mlpData2.predict(X_test_samplesMLP2)

# Compare predictions with actual labels
print("MLP Classifier Predictions:")
for i in range(len(randomSamplesMLP)):
    print(f"Test sample {i+1}: Predicted = {mlpPredictions[i]}, Actual = {y_test_samplesMLP.iloc[i]}")

# Compare predictions with actual labels
print("\nMLP Classifier Predictions:")
for i in range(len(randomSamplesMLP2)):
    print(f"Test sample {i+1}: Predicted = {mlpPredictions2[i]}, Actual = {y_test_samplesMLP2.iloc[i]}")
```

Task 5.1: Evaluation – Confusion Matrix

KNN Classifier Confusion Matrix (80/20 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	28	15
Actual Positive	5	2

KNN Classifier Confusion Matrix (70/30 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	29	6
Actual Positive	11	4

Naive Bayes Classifier Confusion Matrix (80/20 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	37	2
Actual Positive	8	3

Naive Bayes Classifier Confusion Matrix (70/30 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	36	0
Actual Positive	13	1

Support Vector Classifier Confusion Matrix (80/20 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	23	15
Actual Positive	3	9

Support Vector Classifier Confusion Matrix (70/30 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	24	11
Actual Positive	5	10

Multilayer Perceptron Confusion Matrix (80/20 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	34	3
Actual Positive	10	3

Multilayer Perceptron Confusion Matrix (70/30 Split):
Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	40	5
Actual Positive	4	1

Task 5.2: Evaluation – Classification Report

KNN Classification Report 80/20:

	precision	recall	f1-score	support
0	0.83	0.83	0.83	35
1	0.60	0.60	0.60	15
accuracy			0.76	50
macro avg	0.71	0.71	0.71	50
weighted avg	0.76	0.76	0.76	50

KNN Classification Report 70/30:

	precision	recall	f1-score	support
0	0.69	0.76	0.72	33
1	0.43	0.35	0.39	17
accuracy			0.62	50
macro avg	0.56	0.56	0.56	50
weighted avg	0.60	0.62	0.61	50

Naive Bayes Classification Report 70/30:

	precision	recall	f1-score	support
0	0.78	1.00	0.88	35
1	1.00	0.33	0.50	15
accuracy			0.80	50
macro avg	0.89	0.67	0.69	50
weighted avg	0.84	0.80	0.76	50

MLP Classification Report 80/20:

	precision	recall	f1-score	support
0	0.84	0.95	0.89	40
1	0.60	0.30	0.40	10
accuracy			0.82	50
macro avg	0.72	0.62	0.65	50
weighted avg	0.80	0.82	0.80	50

MLP Classification Report 70/30:

	precision	recall	f1-score	support
0	0.78	0.95	0.85	37
1	0.60	0.23	0.33	13
accuracy			0.76	50
macro avg	0.69	0.59	0.59	50
weighted avg	0.73	0.76	0.72	50

SVC Classification Report 80/20:

	precision	recall	f1-score	support
0	0.79	0.65	0.71	34
1	0.45	0.62	0.53	16
accuracy			0.64	50
macro avg	0.62	0.64	0.62	50
weighted avg	0.68	0.64	0.65	50

SVC Classification Report 70/30:

	precision	recall	f1-score	support
0	0.91	0.74	0.82	42
1	0.31	0.62	0.42	8
accuracy			0.72	50
macro avg	0.61	0.68	0.62	50
weighted avg	0.82	0.72	0.75	50

Naive Bayes Classification Report 80/20:

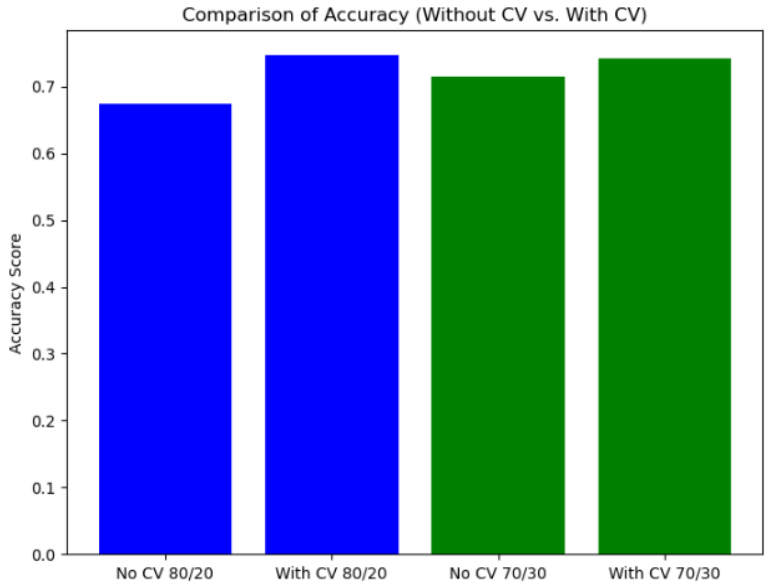
	precision	recall	f1-score	support
0	0.74	0.92	0.82	37
1	0.25	0.08	0.12	13
accuracy			0.70	50
macro avg	0.49	0.50	0.47	50
weighted avg	0.61	0.70	0.64	50

Task 5.3: Evaluation – Fbeta score

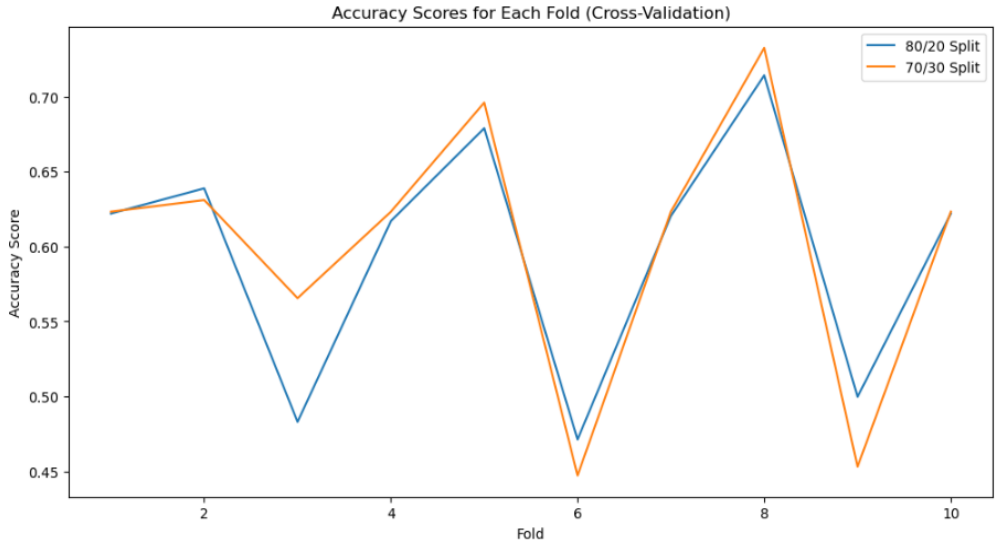
$$F_\beta = \frac{1 + \beta^2}{\frac{\beta^2}{Recall} + \frac{1}{Precision}}$$

```
SVC F1-score 80/20: 0.5263157894736842
SVC F-beta score 80/20: 0.5813953488372092
SVC F1-score 70/30: 0.4166666666666667
SVC F-beta score 70/30: 0.5208333333333334

Naive Bayes F1-score 80/20: 0.11764705882352941
Naive Bayes F-beta score 80/20: 0.0892857142857143
Naive Bayes F1-score 70/30: 0.5
Naive Bayes F-beta score 70/30: 0.3846153846153846

MLP F1-score 80/20: 0.4
MLP F-beta score 80/20: 0.3333333333333333
MLP F1-score 70/30: 0.33333333333333337
MLP F-beta score 70/30: 0.2631578947368421

KNN F1-score 80/20: 0.6
KNN F-beta score 80/20: 0.6
KNN F1-score 70/30: 0.3870967741935484
KNN F-beta score 70/30: 0.3658536585365854
```

Task 6: K-Fold cross-validation




```
Best parameters for 80/20 split: {'C': 10, 'kernel': 'rbf'}
Best parameters for 70/30 split: {'C': 10, 'kernel': 'rbf'}
```



Conclusion